

Let's not even get into the whole computation of AI for enemies and friends—how they move, whether they duck and hide or attack, and about the best path they need to take to be able to attack you! The fact is, games are as complex as programs come today, and are becoming real resource hogs—you need tons of hard disk space, memory, and a 3 GHz+ machine to play the latest titles, not to mention a graphics card that costs in excess of Rs 20,000.

Now imagine if game developers had to code for each and every command, each and every difficulty level, and for each video resolution and effect! And not just code for each operating system, but also for each graphics chipset model! We hope you get the picture: think mass suicides in Silicon Valley and game companies...

APIs

Thankfully, a software coder's life is made easier with the help of Application Programming Interfaces (APIs), which standardise commands that are sent to the hardware and software so a coder just needs to know what command his software is supposed to send, regardless of the make of hardware and software being used.

Think of it as a translator: you want to say something in English to people who only speak a certain dialect of Bengali. The translator is well versed in all dialects of Bengali, and can translate anything you say to any Bengali-speaking person from any part of the globe! That's sort of what an API does—it helps programmers code in “English,” instead of in a different language, depending on the thousands of permutations of hardware and software across the globe.

To elucidate, an API enables both software or game developers as well as hardware manufacturers to standardise the commands, code and drivers, so that life is easier for everyone. Now APIs are used everywhere, even between an OS and a software program. For example, you need to have the Windows API to be able to make a software program for the Windows operating system!

However, this article is not about APIs in general. We're focusing only on the APIs that are used for 3D graphics—whether in graphics software such as Maya or 3ds Max, or 3D games such as *Doom 3*, *F.E.A.R.* and *Half-Life 2*.

A 3D Graphics API consists of everything that comes between a game and the system hardware. This includes the input software, the OS libraries and even the hardware device drivers. So, a 3D API would take commands from a game or graphics program (programmed using that same API, of course) and translate its input requirements and output data so that the hardware (graphics card, CPU, etc.) can understand what's needed of them.

Those who have heard of 3D Graphics APIs will have definitely heard of OpenGL and DirectX (or Direct3D, more appropriately). Let's dig a little deeper into how it all began.

OpenGL

OpenGL started off as Iris GL, a graphics API developed by Silicon Graphics Incorporated (SGI), for the UNIX operating system. In 1992, SGI opened up the API for the public, so people could program using this API, and renamed it OpenGL.

Perhaps the best feature of OpenGL is the way it seamlessly integrates with almost any operating system—from UNIX to Linux, the BSDs, and all versions of Windows.

Remember, OpenGL gets its name from “Open Standard Graphics Library,” and is *not* open source software as many people mistakenly believe. Open Standard means that anyone can license the API and program for it, but no one is allowed to modify it—a stark contrast with Open Source software norms!

OpenGL is developed and maintained under the governance of an Architectural Review Board (ARB) that is made up of companies such as 3DLabs, Apple, ATI, Dell, IBM, Intel, nVidia, SGI and Sun Microsystems. There are more members, but without any voting rights to steer the development of the API, so we won't name them here. Visit www.opengl.org for more information on the partners, and even the minutes of every meeting that has been held thus far!

Perhaps the best feature of OpenGL is the way it seamlessly integrates with almost any operating system—from UNIX to Linux, the BSDs, and all versions of Windows. There's also an official open source implementation of OpenGL called the Sample Implementation (SI), which is completely free for anyone developing open source software or games. Read more about this at www.sgi.com/products/software/opengl/license.html.

OpenGL began, and still is, a low-level graphics library specification. It deals with primitive geometry such as lines, points, simple polygons, images, and bitmaps, to create larger 2D or 3D rendered images. Using OpenGL, programmers can easily use simple commands to render these polygons and shapes by the graphics card's frame buffer. The API was designed to be used with the C and C++ languages, but currently also supports Java, FORTRAN, Ada, and more. For those who don't know what all this is about, don't worry, just take a look at the box *Jargon Buster* to get quick meanings for all the jargon you'll encounter here.

Since OpenGL is independent of the operating system's windowing system, it can be easily used on any platform. A windowing system is a component of the OS that handles things such as window management, colour mapping, and event handling. So basically, OpenGL works on any platform because it interfaces with the windowing system of, say, Windows, or X11 for Linux, and passes commands on to the windowing system. It does the same with the system event handler and the input mechanism. All this results in simple and short codes that are easy to troubleshoot and maintain or change.

Direct3D

Most of you have probably never heard of Direct3D, though a lot of you will have heard about Microsoft's DirectX. DirectX is basically an API suite that contains libraries for audio, input and graphics, all rolled into one. Comparing OpenGL to DirectX, however, is like comparing water to oil, so we'll only talk about the graphics API within DirectX, which is called Direct3D.

The history behind Direct3D is interesting. Back in the days of Windows 95, Microsoft noticed that all game APIs were written with the older Disk Operating System (DOS) in mind. Microsoft wanted to increase native support for games within Windows, which was easier said than done. Windows has many abstraction layers, and